

Relatório — Infraestrutura de Software

Implementação 1: ProcessFlow

Aluno: Mateus Reinaux Batista Meira **E-mail:** mrbrm@cesar.school **Data:** 24/08/2026 **Sistema operacional utilizado:** Windows 11 + WSL2 (Ubuntu, gcc 15.2) **Link do GitHub:** <https://github.com/eFlow-hub/processflow> (privado)

1. Objetivo

O enunciado pede um gerenciador de processos em C (processflow) com modo interativo e modo workflow, capaz de cadastrar tarefas e executá-las de forma simples, sequencial, paralela e em pipeline, com redirecionamento de E/S, mudança de diretório e jobs em background com coleta correta dos filhos. A entrega é o programa completo em processflow.c, com Makefile (alvos padrão, clean e test), seis workflows de teste em testes/, README.md e este relatório, com evidências geradas por script em evidencias.log.

2. Checklist de Requisitos

Requisito do enunciado	Status	Como testar / evidência
Modo interativo com prompt processflow>	OK	./processflow — sessões no evidencias.log
Comando exit encerra o programa	OK	t1_basico.pf; interativo idem
Cadastro de tarefa com task <nome> <programa> [args]	OK	t1_basico.pf
Execução com run <nome>	OK	t1_basico.pf
run sequential t1 t2 t3	OK	t2_grupos.pf; tempo ~3.02s (log 08:14)
run parallel t1 t2 t3	OK	t2_grupos.pf; tempo ~2.01s (log 08:14)
run pipe t1 t2 t3	OK	t4_pipe.pf; saída igual a ls -l sort wc -l no bash
input <tarefa> <arquivo>	OK	t3_redir.pf — sort de nomes.txt bate com o bash
output <tarefa> <arquivo>	OK	t3_redir.pf — resultado.txt truncado
append <tarefa> <arquivo>	OK	t3_redir.pf — duas execuções, arquivo cresce
workdir <diretório>	OK	log 08:14:49 — run onde imprime /tmp
start <tarefa> em background com [jobId] PID	OK	log 08:17:32 — [1] 433, [2] 434
jobs lista os jobs em background	OK	log 08:17:32 — lista só o job ativo
wait <jobId> aguarda job específico	OK	log 08:17:32 — wait 2 bloqueia até o sleep acabar
Coleta correta dos processos filhos (sem zumbis)	OK	log 08:17:32 — ps -el grep defunct vazio com job concluído
Modo workflow com arquivo .pf	OK	make test roda os seis .pf
Impressão de cada linha antes de processar (modo workflow)	OK	qualquer saída de make test
Erro: número incorreto de argumentos → encerra	OK	log 08:14:49 — ./processflow a.pf b.pf → status 1
Erro: arquivo workflow inexistente → encerra	OK	log 08:14:49 — nao_existe.pf → status 1
Erro: tarefa inexistente → continua	OK	t5_erros.pf

Requisito do enunciado	Status	Como testar / evidência
Erro: programa inexistente ou não executável → continua	OK	t5_erros.pf — código 127 informado
Erro: arquivo de I/O não pode ser aberto → continua	OK	t5_erros.pf — filho sai com 1, shell segue
Erro: job inexistente → continua	OK	t5_erros.pf — wait 99
Erro: diretório do workdir inexistente → continua	OK	t5_erros.pf
Linha de comando vazia	OK	t5_erros.pf tem linha em branco; volta ao prompt
Múltiplos espaços em branco	OK	strtok pula delimitadores consecutivos; log 08:12:54
Workflow sem exit / CTRL-D no interativo	OK	t6_sem_exit.pf; EOF tratado como exit implícito
Processos com código de saída diferente de zero	OK	t5_erros.pf — "terminou com código N", sem travar
Processos paralelos terminando fora de ordem	OK	t2_grupos.pf — sleep 2/1/echo; wait na ordem de criação

3. Como Reproduzir

Compilar:

```
make clean
make
```

Executar (modo interativo):

```
./processflow
```

Executar (modo workflow):

```
./processflow testes/t1_basico.pf
```

Rodar os testes:

```
make test
```

4. Arquitetura (resumida)

Arquivos e responsabilidades:

1. processflow.c → todo o programa: parser (strtok), tabela de tarefas, execução (simples/sequential/parallel/pipe), redirecionamento, workdir, jobs em background
2. Makefile → alvos processflow (gcc -Wall -Wextra -g), clean e test
3. testes/*.pf + testes/nomes.txt → workflows de teste cobrindo cada bloco do enunciado

Decisões técnicas:

1. Arquivo único + vetores globais de tamanho fixo (64 tarefas, 64 jobs, 32 args) → mais rápido de escrever e depurar que listas encadeadas → limites documentados; estouro gera mensagem de erro, não corrupção
2. execvp em vez de execl → aceita tanto caminho absoluto quanto nome resolvido pelo PATH → superset do exigido, sem custo
3. Redirecionamento apenas **registrado** na struct da tarefa; open/dup2/close acontecem no filho antes do exec → descritores sobrevivem ao exec, o código do pai não → se o open falha o filho sai com código 1 sem executar o programa
4. waitpid(pid, &st, WNOHANG) sobre a tabela de jobs antes de cada linha lida → filhos de background são colhidos assim que terminam → sem zumbis

5. Estratégias e Diário de Desenvolvimento

5.1 Estratégias

	Nome curto	Contexto	Motivo da troca (se houve)
S1	Arquivo único incremental	Implementação bloco a bloco do guia, um commit por bloco, teste com sessão <code>script</code> a cada bloco	— (não houve troca)
S2			
S3			

5.2 Diário de Tentativas

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Bloco 1: loop, parser, task, run; detecção de <code>exit</code> com <code>strncmp</code> antes do parser	Parcial: funcionou, mas com código duplicado e frágil	<code>exit</code> tratado fora do parser duplicava a tokenização	24/08 08:11	commit f5175e2
2	S1	Refatorar: <code>process_line</code> tokeniza e retorna flag de <code>exit</code>	OK — compila sem warnings, todos os casos do bloco 1 passam		24/08 08:12	log 08:12:54
3	S1	Bloco 2: sequencial (fork+wait alternados) e parallel (forks, depois waits em ordem de criação)	OK — 3.02s vs 2.01s comprova paralelismo		24/08 08:14	log 08:14:01
4	S1	Bloco 3: modo workflow com <code>fopen</code> + eco da linha; <code>workdir</code> com <code>chdir</code>	OK — <code>argc>2</code> e arquivo inexistente encerram com status 1		24/08 08:14	log 08:14:49
5	S1	Bloco 4: <code>open/dup2/close</code> no filho antes do <code>exec</code>	OK — <code>sort</code> bate com o <code>bash</code> ; <code>append</code> cresce; <code>open</code> falho não executa o programa		24/08 08:15	log 08:15:46
6	S1	Bloco 6: pipe com N-1 pipes, fechando todos os fds no pai e nos filhos	OK — mesma contagem do <code>bash</code> , sem travamento		24/08 08:16	log 08:16:31
7	S1	Bloco 7: <code>start/jobs/wait</code> com coleta <code>WNOHANG</code> antes de cada prompt	OK — <code>ps -e1 grep defunct</code> vazio com job já concluído		24/08 08:17	log 08:17:32
8	S1	Validação final: <code>make clean && make && make test</code> com os seis <code>.pf</code>	OK — todos com status 0		24/08 08:18	log 08:18:18
9	S1	<code>make check</code> com golden files: 1ª geração saiu com linhas fora de ordem	Falha: linhas ecoadas do workflow apareciam depois da saída dos filhos	<code>stdout</code> redirecionado usa buffer de bloco; filhos e <code>stderr</code> não passam por ele	24/08 09:10	log 09:10:34
10	S1	<code>fflush(stdout)</code> após ecoar a linha no modo workflow; regenerar esperados	OK — <code>make check</code> passa 2x seguidas (idempotente)		24/08 09:10	log 09:10:34
11	S1	Experimento didático (experimentos/exp1): reproduzi de propósito o filho que dá <code>return</code> em vez de <code>_exit</code> após <code>exec</code> falho	Falha esperada e confirmada: mensagem final do main duplicada — filho sobrevive executando o código do pai; no processflow, <code>_exit(127)</code> evita isso	Filho pós-exec-falho continua no código do pai; <code>return</code> o faz voltar pelo fluxo normal	24/08 09:51	log 09:51:53
12	S1	Experimento didático (experimentos/exp2): <code>open</code>	Falha esperada e confirmada: arquivo nasceu com modo lixo 0011 (-----x--x) —	<code>open</code> é variádico: sem <code>O_CREAT</code> o modo é ignorado;	24/08 10:21	log 10:21:34

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
		com <code>O_CREAT</code> omitindo o 3º argumento, em <code>/tmp (ext4)</code>	do no sem permissão de leitura; com <code>0644</code> explícito, <code>-rw-r--r--</code> . O output do processflow cria <code>0644</code> correto	com ele, lê lixo da pilha se omitido		
13	S1	Experimento didático (experimentos/exp3): pai escreve no pipe e não fecha <code>p[1]</code> antes do <code>waitpid</code>	Falha esperada e confirmada: <code>wc -l</code> nunca recebe EOF e o programa fica pendurado sem erro — <code>timeout 3</code> mata com status <code>124</code> ; no modo certo (<code>close(p[1])</code>) imprime <code>3</code> e termina. <code>run</code> pipe do processflow finaliza sob <code>timeout 10</code>	EOF no pipe só chega quando todas as cópias da ponta de escrita fecham; <code>fork</code> duplica descritores	24/08 12:20	log 12:20:01
14	S1	Experimento didático (experimentos/exp4): zumbi de verdade — filho morre e pai segura o <code>wait</code> ; conferir o transitório do processflow com <code>ps -e1 grep defunct</code>	Parcial: o exp4 mostrou o zumbi (<code>Z+</code> , código <code>42</code> recuperado no <code>wait</code>), mas o <code>ps -e1</code> não achou o transitório do processflow	Coluna CMD do <code>ps -e1</code> truncada no WSL escondeu o <code><defunct></code> ; filtro errado, não ausência de zumbi	24/08 12:32	log 12:32:49
15	S1	Repetir com <code>ps -eo pid,ppid,stat,cmd grep 'Z'</code>	OK — zumbi transitório <code>[echo] <defunct></code> visível enquanto o shell espera a próxima linha; após <code>jobs</code> (coleta <code>WNOHANG</code>), nenhum <code>defunct</code>		24/08 12:34	log 12:34:19
16	S1	Memória sob LeakSanitizer (<code>-fsanitize=address</code> ; sem <code>valgrind</code> no WSL): vazamento proposital em experimentos/exp5 + estresse de 50 redefinições de tarefa (<code>argv</code> , <code>input</code> e <code>output</code> sobrescritos) no processflow	OK — LSan flagrou o leak proposital (9 bytes, pilha do <code>strdup</code>); processflow saiu com status <code>0</code> , sem vazamento: <code>free_task_argv</code> libera as strings antigas antes de reutilizar o slot		24/08 16:35	log 16:35:19
17	S1	<code>wait</code> sem argumento: loop de <code>waitpid</code> específico sobre a tabela de jobs (evitando <code>waitpid(-1)</code>); novo <code>t7_background.pf</code>	OK — 3 jobs coletados, tempo total 2.05s = job mais longo; jobs vazio depois; <code>make check</code> segue verde		24/08 19:09	log 19:09:42
18	S1	Trocar <code>fgets</code> (buffer fixo 1024) por <code>getline</code> ; comparar binário antigo vs novo com linha de 3000 chars	OK — o antigo fatiava silenciosamente (eco truncado + resto virando "comando desconhecido" 2x); o novo processa a linha inteira; ASan confirma <code>free(line)</code> sem vazamento; <code>make check</code> verde		24/08 19:41	log 19:41:29
19	S1	<code>exit</code> avisando jobs ativos + prova do órfão: <code>start</code> de <code>sleep 3</code> e <code>exit</code> imediato	OK — aviso impresso; o <code>sleep</code> sobreviveu ao processflow (adotado pelo <code>init</code>) e sumiu ao terminar, sem zumbi; <code>make clean && make && make test && make check</code> tudo verde		24/08 20:21	log 20:21:27
20	S1	Testes oficiais (testes.tar): saída esperada dos interativos não tem prompt → prompt condicionado a <code>isatty(stdin)</code> ; integração no <code>make check</code>	Parcial: conteúdo idêntico nos 3 testes, mas <code>diff</code> acusou os três	Arquivos <code>-saida.txt</code> do professor sem <code>newline</code> final; nossa saída termina com <code>\n</code> (padrão Unix)	24/08 23:44	log 23:44:19

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
21	S1	Normalizar o esperado no comparador (sed -e '\$a\ ' acrescenta o newline final que falta)	OK — teste1, teste2 (interativos) e teste3 (batch) passam; regressão completa do make check verde		24/08 23:44	log 23:44:45

6. Evidências

6.1 Log automático

O arquivo `evidencias.log` foi gerado com `script -a evidencias.log` e acompanha o `.tar`. Cada sessão inicia com `date`, `whoami` e `pwd`.

6.2 Prints

Erro principal:

Trecho do `evidencias.log` (08:18): tarefa com programa inexistente é informada e o shell continua —

```
processflow: nao foi possivel executar '/bin/programa_que_nao_existe': No such file or directory
tarefa 'ruim' terminou com codigo 127
```

Durante o desenvolvimento o ponto mais delicado foi garantir que falhas do filho (exec/open) usem `_exit` e nunca voltem ao loop do pai.

Validação final:

Trecho do `evidencias.log` (08:18): `make test` executa os seis workflows com status 0, incluindo `t5_erroes.pf` terminando em `ainda-vivo` após sobreviver a todos os erros.

7. Uso de IA

Onde usei IA (2 a 3 linhas):

Utilizei o Claude Code (modelo Fable 5) como assistente de implementação: escrita do código C bloco a bloco seguindo o guia da disciplina, criação dos testes `.pf`, do Makefile e a condução das sessões de teste no WSL que geraram o `evidencias.log`.

Prompts principais (liste, sem história):

1. "Vamos começar essa atividade de acordo com os mds anexados" (guia + template do relatório)
2. Implementação incremental dos blocos 1–8 do guia (parser/task/run, sequencial/parallel, workflow/workdir, redirecionamento, pipe, jobs)
3. Geração das sessões de teste com `script` e validação de cada bloco

O que validei manualmente e como:

Rodei `make clean && make && make test` no WSL; comparei `run pipe` com `ls -l | sort | wc -l` e o `sort` redirecionado com `sort nomes.txt` direto no bash; conferi os tempos sequencial (~3s) vs paralelo (~2s) e a ausência de zumbis com `ps -e1 | grep defunct`.

8. Reflexão Final

A decisão de manter tudo em um arquivo com vetores fixos acelerou muito o ciclo escrever-testar-commatar, e o custo (limites fixos) é irrelevante para o escopo. A regra mais valiosa do guia foi "feche tudo que não vai usar"

no pipe: seguida à risca desde o início, o pipe funcionou de primeira, sem o travamento clássico. O único retrabalho real foi a detecção do `exit`, que comecei fora do parser e tive que trazer para dentro. Testar cada bloco numa sessão `script` logo após implementá-lo deixou as evidências prontas de graça, em vez de virar uma caça no fim. Na próxima, começaria já com o `report_status` de códigos de saída, que acabou sendo útil em quase todos os testes de erro.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	Sim
Rodei os testes e <code>evidencias.log</code> foi gerado	Sim
Tenho os prints obrigatórios	Parcial — trechos do log no relatório; screenshots a critério do aluno
Testei pelo menos um caso limite e um caso inválido	Sim (<code>t5_erros.pf</code> , <code>t6_sem_exit.pf</code>)
Preenchi a seção de uso de IA	Sim
Revisei o relatório e removi frases genéricas / vazias	Sim
Diretório e <code>.tar</code> nomeados com as iniciais do e-mail em minúsculas	Sim (<code>mrbm</code>)
<code>evidencias.log</code> está dentro do <code>.tar</code>	Sim

10. Se eu tivesse mais 2 horas

Adicionaria suporte a argumentos com aspas no parser, para permitir argumentos com espaços. Trocaria a coleta de background por tratamento de `SIGCHLD`, eliminando a janela de zumbi transitório documentada no diário #15. Tornaria o `t4_pipe.pf` determinístico usando entrada fixa (`cat` de arquivo em vez de `ls -l`), permitindo incluí-lo no `make check`. (Vários itens da versão original desta lista acabaram entrando na entrega ao longo do dia: `make check` com golden files — #9/#10 —, verificação de memória com `LeakSanitizer` — #16 —, `wait` sem argumento — #17 —, `getline` no lugar do buffer fixo — #18 — e o `exit` com aviso de jobs ativos — #19.) (O `make check` com golden files, que estava nesta lista, acabou entrando na entrega — diário #9 e #10 — e de quebra revelou um bug real de buffering no eco do modo workflow.)